

POPScore: A Multi Model Deep Learning System for Predicting Song Popularity and Suggesting Audio Enhancements

Arad Harush 211879606 • Shahar Nahum 323964817

CODE: 26-1-D-5

Dr. Dan Lemberg • Mrs. Elena Kramer

Background & The Problem:

The music industry is highly subjective and saturated. Today, commercial success is often driven by an artist's established brand reputation and marketing budget rather than the pure musical quality of the track itself. This creates a significant barrier for independent artists and new producers who rely solely on their sound. Furthermore, when a track doesn't succeed, creators lack objective, data-driven feedback to understand why the audio failed to resonate or how to fix it technically in the studio before release.

The Need:

There is a critical need for an objective, AI-driven "virtual producer." Creators need a tool that can strip away the bias of fame, evaluate a song strictly based on its acoustic and lyrical DNA, and provide concrete, actionable mixing and mastering recommendations to actively increase the track's potential for commercial success.

Our Solution & Technologies

PopScore is a two-step AI pipeline: an objective "Judge" that scores a song's hit potential, and a "Virtual Producer" that suggests mix adjustments to maximize that score.

Audio & Text Processing: Librosa extracts audio features and Mel-spectrograms. SBERT (Sentence-BERT) encodes lyrical semantics.

Predictive Model (PyTorch): A custom dual-branch Deep Learning model using a ResNet50 CNN to analyze spectrograms alongside textual embeddings.

Optimization Engine (Reinforcement Learning): A PPO Agent (Stable-Baselines3) that treats the song's feature vector as a latent environment, learning to recommend optimal studio adjustments (Loudness, Energy, etc.).

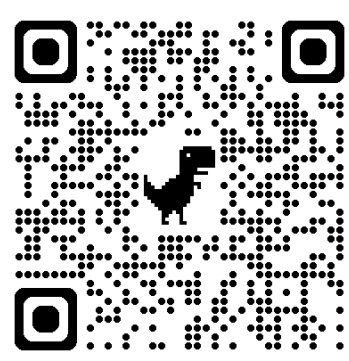
Deployment & UI: A dynamic web interface built with Gradio, featuring visual analytics (Spider charts via Matplotlib), with models hosted on the Hugging Face Cloud.

Try It Yourself

Git Repository

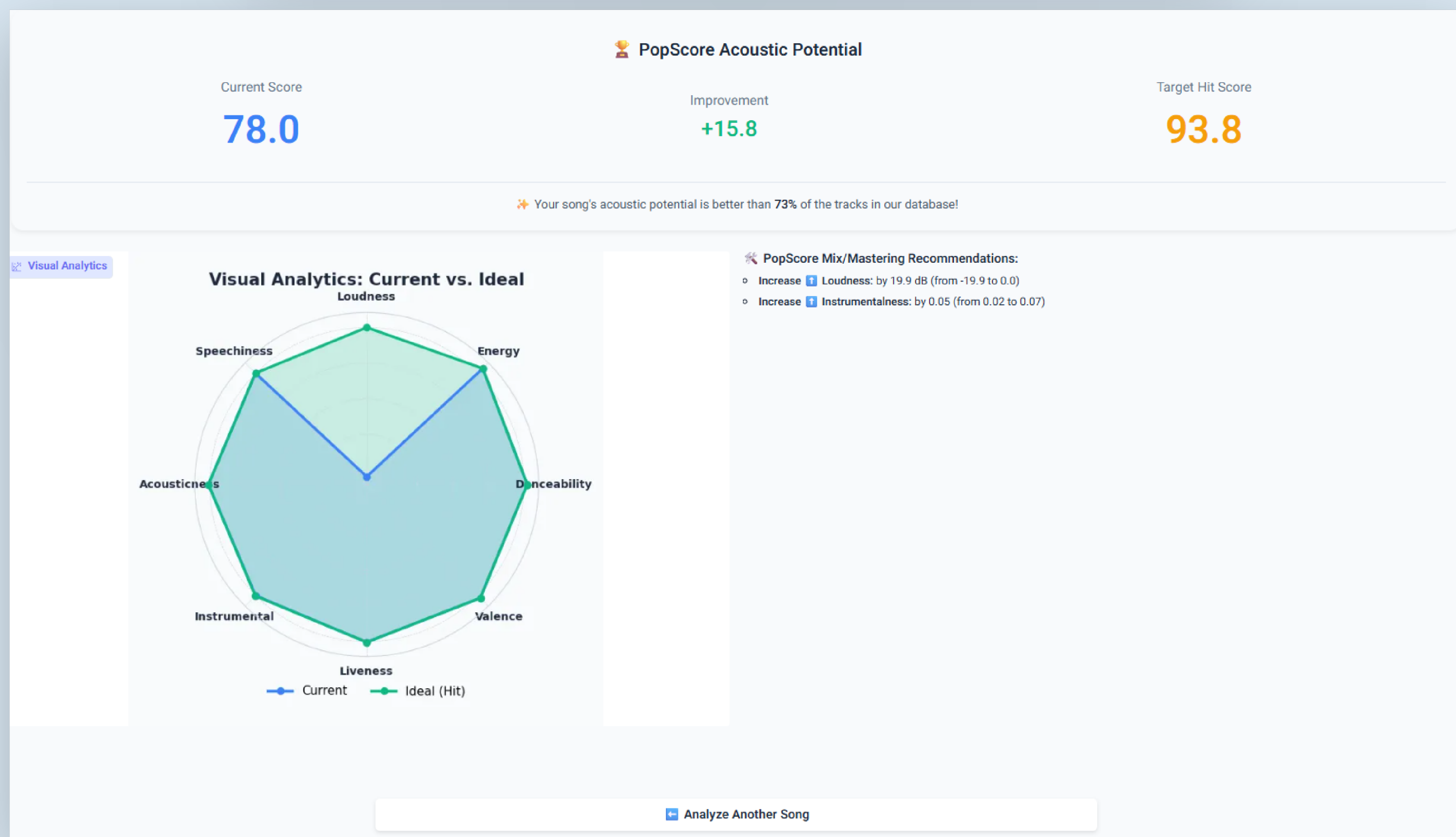
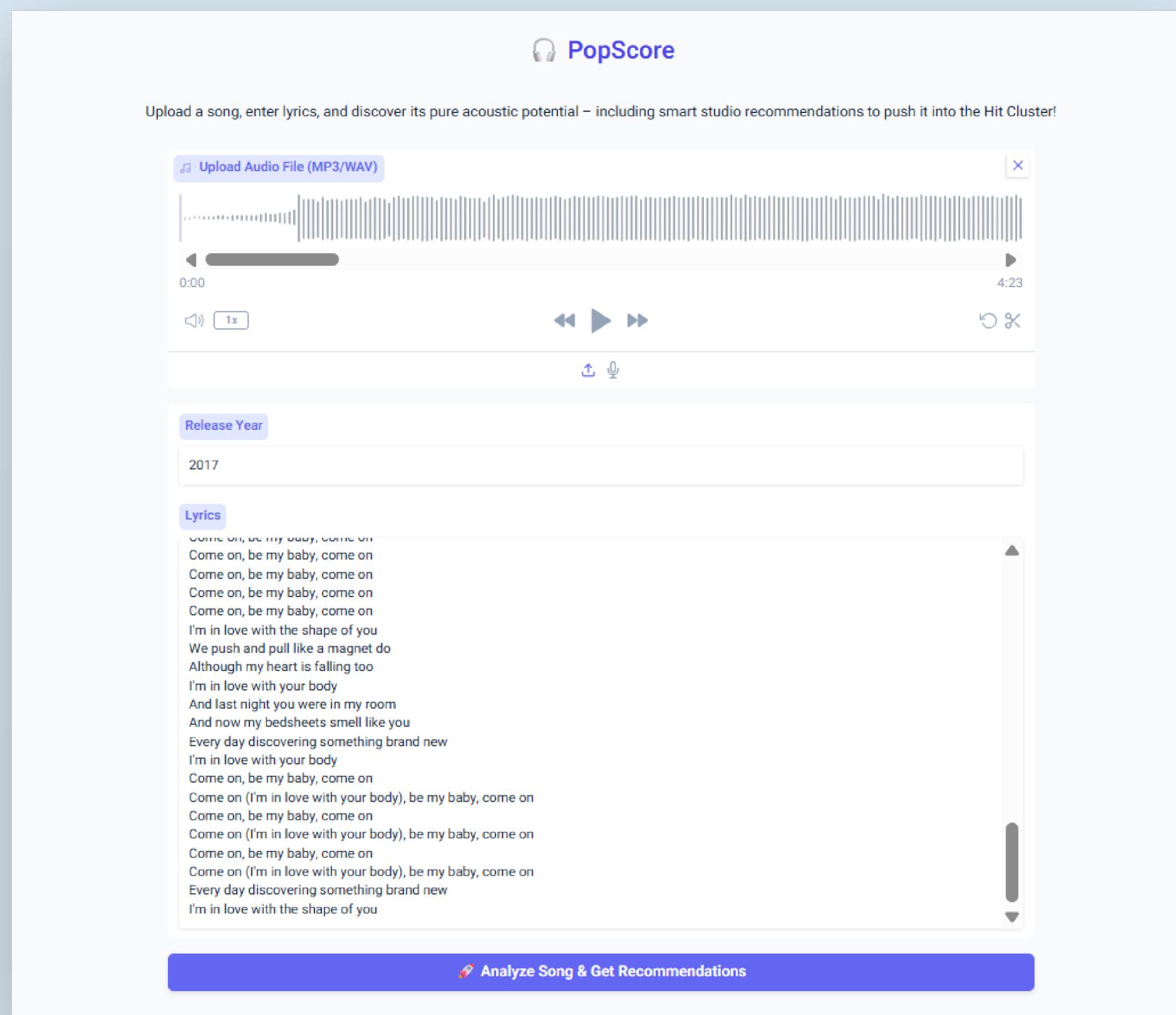


Run App



Functional Requirements

- The system will allow users to upload audio files (e.g., MP3, WAV), input song lyrics and release year.
- The system will convert raw audio into spectrogram representations and tokenize lyrics for NLP processing.
- The system will calculate and display a predicted popularity score (0-100) based on the processed inputs.
- Based on the analysis, the system will provide textual recommendations for improving the song's commercial potential.



Key Metrics & Achievements

Optimization Success Rate: The Reinforcement Learning (PPO) agent successfully finds an optimized "Hit Target" state for 100% of tested tracks.

Average Score Improvement: On average, the AI producer's recommendations improve a song's predicted popularity score by +5.74 points.

Real-time Processing: The entire pipeline—from raw .mp3 ingestion, spectrogram generation, text embedding, to RL optimization—completes in under 5 seconds, providing instant feedback to the user.

System Deployment: Successfully deployed a fully functional, cloud-hosted web application allowing users to interact with the models seamlessly.

Prediction Accuracy (MAE): The Deep Learning hit-predictor achieved a Mean Absolute Error of 19.06, demonstrating a strong ability to accurately evaluate pure acoustic potential without relying on artist fame.

Key Challenges

The "Fame Bias" (Data Constraint): Real-world popularity metrics (e.g., Spotify scores) are heavily skewed by an artist's fame. The major challenge was architecting the model to ignore metadata entirely and learn to score solely based on raw audio and lyrics.

Multimodal Data Fusion: Successfully combining completely different data structures—2D Mel-spectrograms (vision), text embeddings (NLP), and numerical features—into a single, synchronized PyTorch pipeline required extensive architectural tuning.

Constraining the RL Agent: Training the PPO agent to suggest musically realistic changes was challenging. We had to build a custom "Surrogate Model" and implement strict boundaries so the AI wouldn't recommend impossible studio adjustments (e.g., capping tempo or energy limits).

Compute & Latency Constraints: Running heavy audio processing (Librosa) alongside multiple deep learning models simultaneously required significant code optimization to ensure a fast, real-time user experience on cloud-hosted environments.